DS性能优化汇总

一、网络同步

1. ReplicationGraph

（1）分块同步

ReplicationGraph是UE引擎层提供的一个网络分块同步优化的技术方案(PUBG是自己实现了一个九宫格分块同步算法，算法原理类似)，可以减少同步量，降低DS的同步开销

（2）应用代码

```
/**
 * GameReplicationGraph
 **/
UCLASS(transient, config = Engine)
class UGameReplicationGraph :public UBasicReplicationGraph
{
    GENERATED_BODY()

public:
    UGameReplicationGraph();

```

```
    virtual void InitGlobalActorClassSettings() override;
    virtual void InitGlobalGraphNodes() override;

protected:
    UPROPERTY(Config)
        TArray<FRepListSizeLevel> RepListSizeConfigs;

    UPROPERTY(Config)
        float GridCellSize = 10000.f;
};
```

（3）展望

目前我们联机地图还没涉及大世界地图的规模，地图不够大，因此RG的优化有一定效果，但不是特别明显

2. AbilityAttributes同步

（1）FastArray

UE提供的一种优化同步数组的数据结构，可以做差量同步，但就算这样AbilityAttributes同步量依然很高

（2）主要优化内容

- **合并同步**

修改FastArray同步机制，加一个缓冲合并同步的处理算法

```
/*
 * FAbilityAttributeArray
 * 技能角色属性自定义同步数组
 *
 * 数据同步更新:
 * (1) MarkItemDirty: 会及时写入网络同步buffer中，同步量较大，不推荐用此方式（除非是需要及时同步的数据）
 * (2) MarkItemToBeReplicated: 通过网络同步时间间隔控制，合并一段时间的数据更新，减少网络同步量对网络同步更友好，优先采用此方式
 */
USTRUCT(BlueprintType)
struct FAbilityAttributeArray : public FFastArraySerializer {
}
```

- **同步item和非同步item分离**

```cpp
USTRUCT(BlueprintType)
struct FAbilityAttributeArray : public FFastArraySerializer {
    ...
    bool NetDeltaSerialize(FNetDeltaSerializeInfo& DeltaParams) {
        // 只需要同步RepAttributes
        return FastArrayDeltaSerialize<FAbilityAttributeItem, FAbilityAttributeArray>(RepAttributes, DeltaParams, *this);
    }
    ...
private:
    ...
    // 同步item
    UPROPERTY(Transient)
    TArray<FAbilityAttributeItem> RepAttributes;

    // 非同步item(仅DS端使用)
    UPROPERTY(Transient)
    TArray<FAbilityAttributeItem> NotRepAttributes;
    ...
};
```

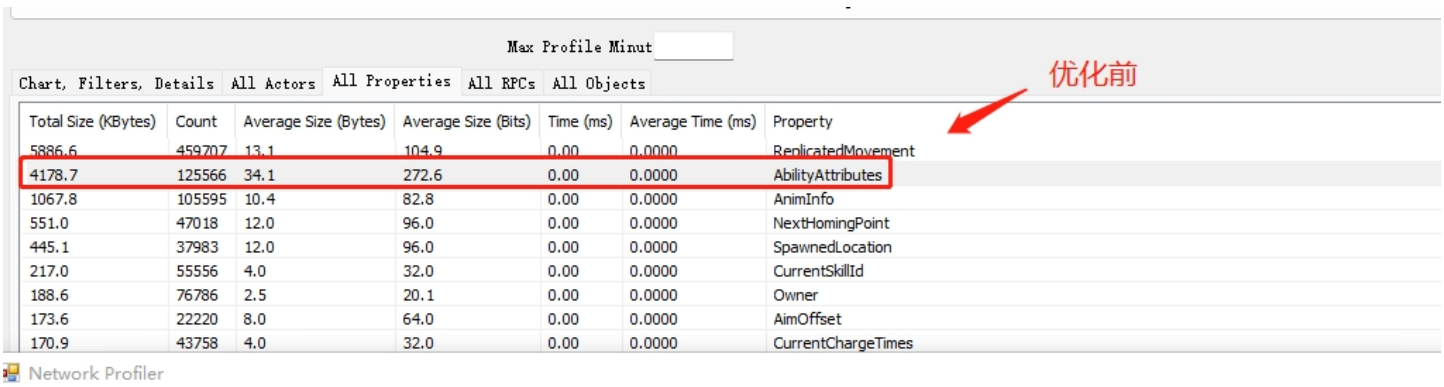
- **同步去重，减少冗余同步**

  重载==，比较当前状态和最新状态是否存在差异

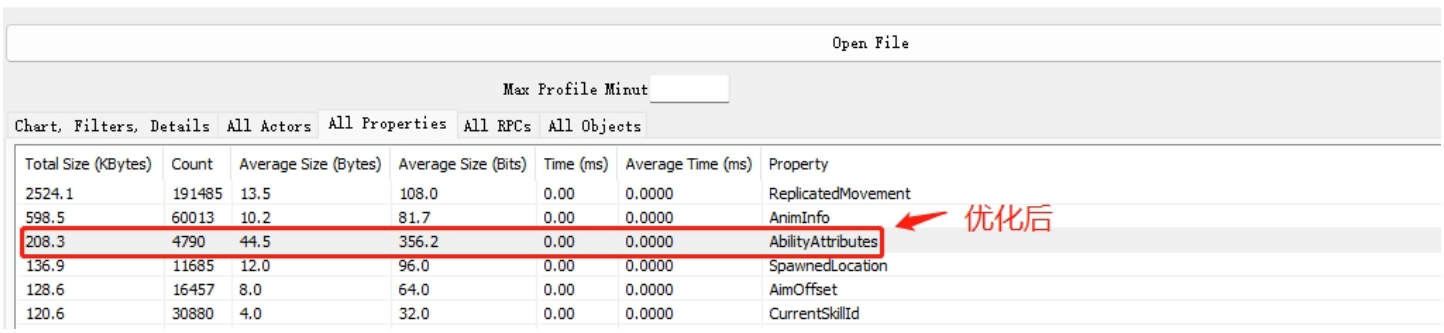
- **去掉无需同步的Item属性**

  不需要同步的Item属性选择*NotReplicated* => 段

（3）优化效果



Total Size (KBytes)	Count	Average Size (Bytes)	Average Size (Bits)	Time (ms)	Average Time (ms)	Property
5886.6	459707	13.1	104.9	0.00	0.0000	ReplicatedMovement
4178.7	125566	34.1	272.6	0.00	0.0000	AbilityAttributes
1067.8	105595	10.4	82.8	0.00	0.0000	AnimInfo
551.0	47018	12.0	96.0	0.00	0.0000	NextHomingPoint
445.1	37983	12.0	96.0	0.00	0.0000	SpawnedLocation
217.0	55556	4.0	32.0	0.00	0.0000	CurrentSkillId
188.6	76786	2.5	20.1	0.00	0.0000	Owner
173.6	22220	8.0	64.0	0.00	0.0000	AimOffset
170.9	43758	4.0	32.0	0.00	0.0000	CurrentChargeTimes



Total Size (KBytes)	Count	Average Size (Bytes)	Average Size (Bits)	Time (ms)	Average Time (ms)	Property
2524.1	191485	13.5	108.0	0.00	0.0000	ReplicatedMovement
598.5	60013	10.2	81.7	0.00	0.0000	AnimInfo
208.3	4790	44.5	356.2	0.00	0.0000	AbilityAttributes
136.9	11685	12.0	96.0	0.00	0.0000	SpawnedLocation
128.6	16457	8.0	64.0	0.00	0.0000	AimOffset
120.6	30880	4.0	32.0	0.00	0.0000	CurrentSkillId

3. UnmappedObjects

（1）问题

前同事针对Actor和Owner之间存在强依赖关系的，将Owner的NetGuid映射Bunch中，保证Owner先同步到客户端处理

这种做法一方面会增加客户端处理UnmappedObjects的开销(特别是频繁创建的Skill Actor)，另一方面也会增加同步量

（2）优化方案

- 直接去掉Actor Owner的映射机制
- 业务层做好判空处理 & 必要逻辑可以写在*OnRep_Owner*回调函数中

```
void UWaitOwnerActorChannel::AppendMustBeMappedGuids(FOutBunch* Bunch) {
    // // Actor第一次同步到客户端时，将Owner的NetGuid映射到Bunch中，客户端会等待Owner的Bunch到了之后再处理自身
    // if ((Bunch->bOpen) && Actor && Actor->GetOwner() && !Actor->GetOwner()->IsA<AController>()) {
    //     UPackageMapClient* PackageMapClient = CastChecked< UPackageMapClient >(Connection->PackageMap);
    //     TArray< FNetworkGUID >& MustBeMappedGuidsInLastBunch = PackageMapClient->GetMustBeMappedGuidsInLastBunch();
    //     const FNetworkGUID OwnerNetGuid = PackageMapClient->GetNetGUIDFromObject(Actor->GetOwner());
    //
    //     MustBeMappedGuidsInLastBunch.Add(OwnerNetGuid);
    //     Super::AppendMustBeMappedGuids(Bunch);
    //
    //     return;
    // }

    Super::AppendMustBeMappedGuids(Bunch);
}
```

二、移动同步

1. 子弹移动优化

（1）Player子弹

Bullet实体对象改为射线子弹，去掉角色bullet的创建和同步 => 段

（2）Monster子弹

1> 初步优化

- 重构子弹移动组件逻辑, 使用引擎层UProjectileMovementComponent(抛体移动组件)
- 模拟端插值平滑 & 模拟补偿
- DS端降低同步频率

```
/**
 * 抛体移动组件基类
 * - 包含引擎层ProjectileMovementComponent的所有功能
 * - 加入客户端插值算法(优化客户端平滑表现)
 * - 加入客户端模拟补偿算法(可减少Actor移动同步且不影响客户端表现, 包括弱网下的较极端的情况)
 */
UCLASS()
class GAME_API UGameProjectileMovementComponent : public UProjectileMovementComponent {
    GENERATED_BODY()

public:
    /**
     * 是否启动模拟补偿.
     * - 在插值平滑算法处理结束且未收到DS最新状态的时间段内, 进行模拟补偿,
     *   以减少客户端与DS的状态差异, 提升客户端表现.
     */
    UPROPERTY(EditAnywhere, BlueprintReadWrite, Category=ProjectileInterpolation)
    bool bSimulateCompensation;

    UGameProjectileMovementComponent(const FObjectInitializer& ObjectInitializer);

    virtual void TickComponent(float DeltaTime, ELevelTick TickType, FActorComponentTickFunction* ThisTickFunction) override;

protected:
    // 插值平滑
    virtual void TickInterpolation(float DeltaTime) override;

    // 模拟补偿
    virtual void SimulateMoveOnClientSide(float DeltaTime);
};

void UGameProjectileMovementComponent::TickComponent(float DeltaTime, ELevelTick TickType, FActorComponentTickFunction* ThisTickFunction) {
    Super::TickComponent(DeltaTime, TickType, ThisTickFunction);
```

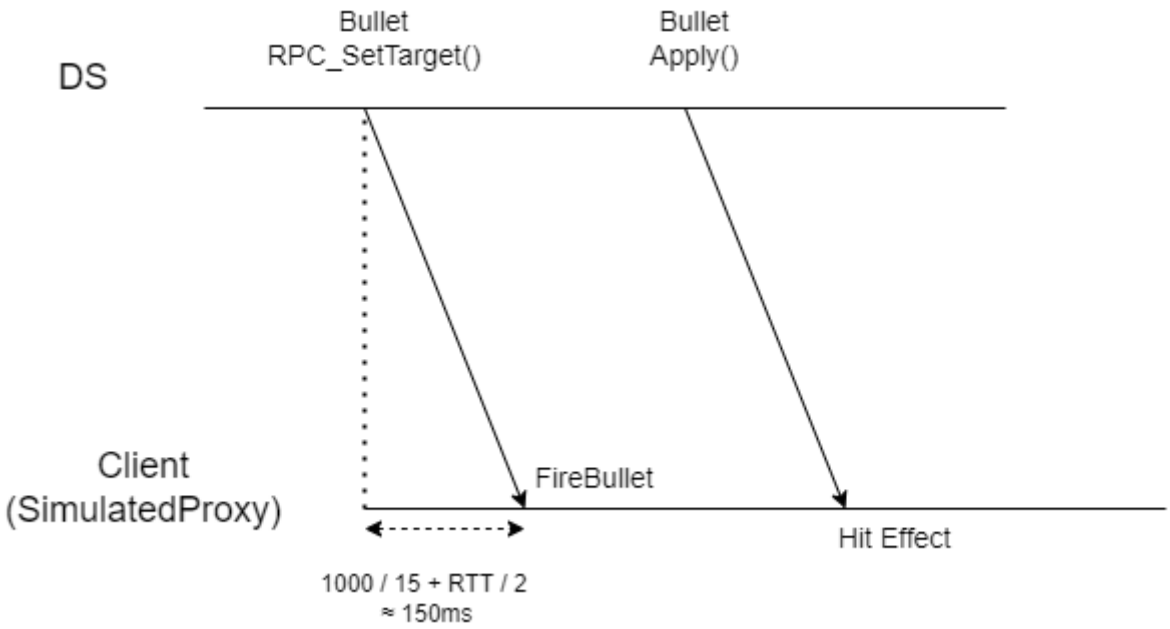
```

    // 插值结束后启动模拟补偿
    if (bInterpMovement &&
        bSimulateCompensation &&
        bInterpolationComplete && IsNetMode(NM_Client)) {
        SimulateMoveOnClientSide(DeltaTime);
    }
}

void UGameProjectileMovementComponent::TickInterpolation(float DeltaTime) {
    if (!bInterpolationComplete) {
        if (USceneComponent* InterpComponent = GetInterpolatedComponent()) {
            const float InterpMoveDistance = Velocity.Size() * DeltaTime;
            const float LeftInterpMoveDistance = InterpLocationOffset.Size();
            // 位移插值
            if (InterpMoveDistance < LeftInterpMoveDistance) {
                InterpLocationOffset = InterpLocationOffset.GetSafeNormal() * (1 - InterpMoveDistance / LeftInterpMoveDistance);
            } else {
                InterpLocationOffset = FVector::ZeroVector;
            }

            // 旋转插值
            if (DeltaTime < InterpRotationTime && bInterpRotation) {
                InterpRotationOffset = FQuat::FastLerp(InterpRotationOffset, FQuat::Identity, DeltaTime / InterpRotationTime).GetNormalized();
            } else {
                InterpRotationOffset = FQuat::Identity;
            }
            ...
        } else {
            ResetInterpolation();
            bInterpolationComplete = true;
        }
    }

    if (bInterpolationComplete && bAutoUpdateTickRegistration && (UpdatedComponent == nullptr)) {
        UpdateTickRegistration();
    }
}

void UGameProjectileMovementComponent::SimulateMoveOnClientSide(float DeltaTime) {
    ...
    // 移动模拟补偿
    const FVector& CurrentLocation = UpdatedComponent->GetRelativeLocation();
```

```
    const FVector& CurrentVelocity = RepMovement.LinearVelocity;
    const FVector& NewLocation = CurrentLocation + CurrentVelocity * DeltaTime;

    // 旋转模拟补偿
    const FRotator& CurrentRotation = UpdatedComponent->GetRelativeRotation();
    const FVector& CurrentAngularVelocity = RepMovement.AngularVelocity;
    const FRotator& NewRotation = CurrentRotation + CurrentAngularVelocity.Rotation() * DeltaTime;

    UpdatedComponent->SetRelativeLocationAndRotation(NewLocation, NewRotation);
}
```

2> 进一步优化

- DS统一发射Bullet并同步初始位置到各个模拟端
- 模拟端根据DS同步相同的初始状态，并用相同的计算过程进行模拟子弹移动
- 模拟端只模拟表现，不参与逻辑计算(碰撞 / 伤害)
- DS端关闭子弹移动同步
- DS最终同步Bullet Apply的效果

```
void ABullet::OnComponentBeginOverlap(UPrimitiveComponent* OverlappedComp, AActor* Other,
    ...

    // DS端Apply，然后同步结果到模拟端
    if (HasAuthority() != false) {
        Apply(Other, Hit);
    }

    ...
}
```



（3）优化效果

优化前

Actor	MS	KB/s	Bytes	Count	Update HZ	Rep HZ	Waste
BP_Girl004	3278.07	0.60	808663	57930	44.09	28.19	36.05
BP_Girl004_Prop02_2_New	985.75	0.52	700116	51701	39.35	24.17	38.58
FireBullet	1398.86	0.51	684932	28857	21.96	21.96	0.00
CastSkill	8336.09	0.51	680143	381249	290.14	55.15	80.99
BP_Girll005a	3840.67	0.50	677463	64764	49.29	22.74	53.86
BP_Girl007	3160.81	0.28	375298	60772	46.25	13.57	70.65

Property	Bytes
ReplicatedMovement	392728
SpawnedLocation	187500
CurrentBulletID	62500
Owner	25127
RemoteRole	5864
Role	5864

Summary Actors Token Details

Actor	MS	KB/s	Bytes	Count	Update HZ	Rep HZ	Waste
Bullet_Girl004_Skill03_011	175.74	0.26	350769	26349	20.05	20.05	0.00
Bullet	242.05	0.24	319313	18099	13.77	13.77	0.00
BP_Girl002a	2548.40	0.20	269518	56686	43.14	9.89	77.08
Trap_Boss_003_Skill07_01	1085.03	0.16	212134	114765	87.34	7.45	91.47
Bullet_Girl004_Skill03_012	108.11	0.15	206371	16323	12.42	12.42	0.00
BP_Boss_003	1492.69	0.15	205029	29266	22.27	10.92	50.97
BP_Girl003	2522.33	0.13	174810	54832	41.73	11.23	73.08

Property	Bytes	Co
ReplicatedMovement	256212	18
SpawnedLocation	36588	30
CurrentBulletID	12196	30
Owner	8979	30
RemoteRole	1652	66
Role	1652	66
bIsActive	1029	82

优化后

Actor	MS	KB/s	Bytes	Count	Update HZ	Rep HZ	Waste
BP_Mon_017	182.69	0.12	107720	10923	12.91	8.69	32.68
CastSkill	2035.74	0.10	88830	144039	170.25	22.99	86.50
Effect_girl012_Skill01_01	257.26	0.09	81425	190836	225.57	1.73	99.23
Bullet	199.77	0.09	77311	10782	12.74	8.23	35.41
GameBaseState	115.66	0.09	74743	18346	21.68	21.68	0.00
BP_Mon_004c	119.23	0.08	67413	7431	8.78	5.88	33.06
BP_Mon_003a	89.20	0.07	62286	5445	6.44	4.61	28.43
BP_Mon_005	109.68	0.07	60756	6603	7.80	5.12	34.44
BP_Mon_104c	110.54	0.07	59131	6273	7.41	4.44	40.08
BP_Mon_112	130.92	0.07	57315	8490	10.04	4.65	53.71
BP_Mon_016	77.46	0.06	49904	4634	5.48	3.98	27.34
BP_Wp31401	144.93	0.06	48506	36675	43.35	2.07	95.23

Property	Bytes	Co
SpawnedLocation	41544	34
CurrentBulletID	13848	34
Owner	9072	34
AbilityOwner	9072	34
RemoteRole	1461	58
Role	1461	58
bIsActive	854	68

2. 角色移动优化

（1）问题

- 角色静止状态的移动包同步频率极高，并包太少
- 角色运动状态的移动包同步频率过高，并包太少

以上均导致DS端ServerMovePacked RPC处理耗时较高

（2）优化方案

- 引擎层

PendingMove的延迟时间上限和下限改到ini配置

```
class ENGINE_API AGameNetworkManager : public AInfo
{
    GENERATED_UCLASS_BODY()

    ...

    /** ClientMinNetSendMoveDeltaTime is used clamped ClientNetSendMoveDeltaTime */
    UPROPERTY(GlobalConfig)
    float ClientMinNetSendMoveDeltaTime;

    /** ClientMaxNetSendMoveDeltaTime is used clamped ClientNetSendMoveDeltaTime */
    UPROPERTY(GlobalConfig)
    float ClientMaxNetSendMoveDeltaTime;

    ...
};

void UCharacterMovementComponent::ReplicateMoveToServer(float DeltaTime, const FVector& NewAcceleration)
{
    ...

    // Add NewMove to the list
    if (CharacterOwner->IsReplicatingMovement())
    {
        ...
        if (bCanDelayMove && ClientData->PendingMove.IsValid() == false)
        {
            // Decide whether to hold off on move
```

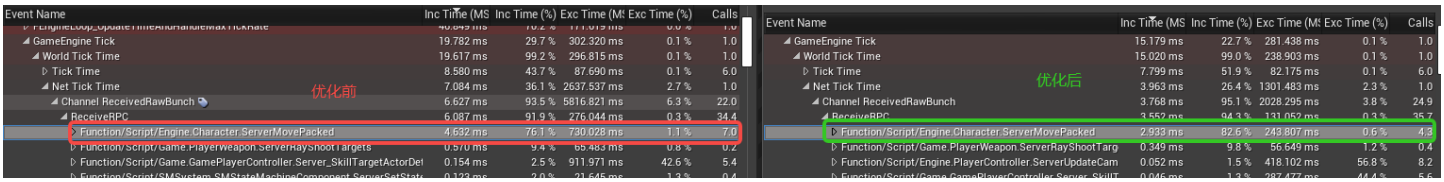
```
            const AGameNetworkManager* GameNetworkManager = (const AGameNetworkManager*)(AGameNetworkManager::StaticClass()->GetDefaultObject());
            const float NetMoveDelta = FMath::Clamp(GetClientNetSendDeltaTime(PC, ClientData, NewMovePtr), GameNetworkManager->ClientMinNetSendMoveDeltaTime, GameNetworkManager->ClientMaxNetSendMoveDeltaTime);

            if ((MyWorld->TimeSeconds - ClientData->ClientUpdateTime) * MyWorld->GetWorldSettings()->GetEffectiveTimeDilation() < NetMoveDelta)
            {
                // Delay sending this move.
                ClientData->PendingMove = NewMovePtr;
                return;
            }
        }
    }
    ...
}
```

- 项目层
  - 增加SendMove的最大上限延迟时间
  - 增加运动时SendMove延迟时间
  - 静止时SendMove的延迟时间设为最大上限

保证PendingMove合并最大化，优化客户端流量的同时优化DS端处理Move的耗时

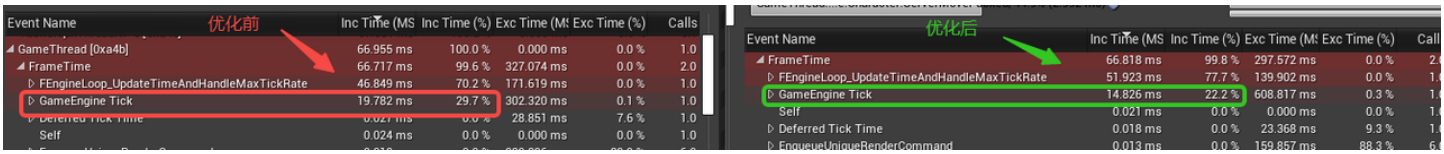
```
[/Script/Engine.GameNetworkManager]
;主控端同步SendMove的最大延迟时间(默认0.2)
ClientMaxNetSendMoveDeltaTime=1

;主控端同步SendMove的延迟时间
ClientNetSendMoveDeltaTime=0.066            // (PlayerCount < 10), 和DS更新频率一致
ClientNetSendMoveDeltaTimeThrottled=0.066    // (PlayerCount >= 10), 和DS更新频率一致

;主控端角色静止时同步SendMove的延迟时间
ClientNetSendMoveDeltaTimeStationary=1
```

（3）优化效果



[优化前]　　[优化后]

4.632ms => 2.933ms



三、动画

1. 主控端动画和移动同步合并

（1）同步机制

ServerRPC　　Replication

AutonomousProxy ======> Authority(DS) ======> SimulateProxy

（2）实现代码(部分)

```
/*
 * 主控端玩家移动数据
 * - Client -> Server
 * - 动画同步数据和移动同步数据整合在一起，方便优化
 */
struct FCustomCharacterMoveData : public FCharacterNetworkMoveData
{
public:
    virtual void ClientFillNetworkMoveData(const FSavedMove_Character& ClientMove, ENetworkMoveType MoveType) override;

    ...

    // 动画数据合并
    uint8 bHasAnimInfoNet;
    FAnimInfoNet AnimInfoNet;

    ...
};

void FCustomCharacterMoveData::ClientFillNetworkMoveData(const FSavedMove_Character& ClientMove, ENetworkMoveType MoveType) {
```

```
    Super::ClientFillNetworkMoveData(ClientMove, MoveType);

    ...

    // 动画数据合并
    if (UGameCharacterAnimInstance* AnimInstance = Cast<UGameCharacterAnimInstance>(CharacterOwner->GetGameAnimInstance())) {
        const FAnimInfoNet& NewAnimNetInfo = CharacterOwner->GetAnimInfo();
        bHasAnimInfoNet = (NewAnimNetInfo != AnimInfoNet);
        if (bHasAnimInfoNet) {
            AnimInfoNet.CopyData(NewAnimNetInfo);
            AnimInfoNet.TimeStamp = TimeStamp;
        }
    }
}
```

（3）合并同步

- 解决一些因动画和移动状态不同步导致模拟端表现异常的问题
- 减少相同属性的同步，优化流量开销

2. 骨骼更新优化

（1）动画Tick更新选项

```
/** Skinned Mesh Animation Tick option based on rendered or not. This dictates "TickPose and RefreshBoneTransforms" */
UENUM(BlueprintType)
enum class EVisibilityBasedAnimTickOption : uint8
{
   /** Always Tick and Refresh BoneTransforms whether rendered or not. */
   AlwaysTickPoseAndRefreshBones,
   /** Always Tick, but Refresh BoneTransforms only when rendered. */
   AlwaysTickPose,
   /**
      When rendered Tick Pose and Refresh Bone Transforms,
      otherwise, just update montages and skip everything else.
      (AnimBP graph will not be updated).
   */
   OnlyTickMontagesWhenNotRendered,
   /** Tick only when rendered, and it will only RefreshBoneTransforms when rendered. */
   OnlyTickPoseWhenRendered,
};
```

（2）优化思路

引擎默认选项*AlwaysTickPoseAndRefreshBones*，这会导致骨骼每帧都会更新，目前改成AlwaysTickPose，然后提供api根据需要打开骨骼更新

```
/*
 * DS端性能优化接口：动态开启/关闭骨骼更新
 */
void UGameLibrary::SetAnimationTickEnableOnDS(const AActor* InActor, const bool bEnable) {
    if (!InActor || !InActor->IsNetMode(NM_DedicatedServer)) return;

    ...

    SkeletalMeshCmp->VisibilityBasedAnimTickOption = bEnable ?
        EVisibilityBasedAnimTickOption::AlwaysTickPoseAndRefreshBones:
        EVisibilityBasedAnimTickOption::AlwaysTickPose;
}
```

（3）优化效果

优化前

优化后

四、对象池

1. RepList Pool

（1）问题

RepList Pool用于存储DS端同步Actor时需要保存的相关信息，引擎默认pool的size比较小，**当Actor同步量增加后，会导致RepList Pool扩容(memory copy)**，开销较高(200ms+，最高甚至能到1000ms+)

（2）优化

RepList Pool分四档，按照缓存档位分配，目前最高档是能满足现有需求

```
[/Script/Game.GameReplicationGraph]
+RepListSizeConfigs=(ListSize=3, NumLists=12)
```

```
+RepListSizeConfigs=(ListSize=6, NumLists=12)
+RepListSizeConfigs=(ListSize=128, NumLists=64)
+RepListSizeConfigs=(ListSize=256, NumLists=12)
```

```
FPool& GetOrCreatePoolForListSize(int32 ExpectedMaxSize, bool ForPreAllocation=false)
{
    // 按照缓存档位分配
    FPool* Pool = PoolTable.FindByPredicate([&ExpectedMaxSize](const FPool& InPool) { return ExpectedMaxSize <= InPool.ListSize; });
    if (!Pool)
    {
        ...
        Pool = new (PoolTable) FPool( ExpectedMaxSize );
    }
    return *Pool;
}
```

2. 对象池系统

（1）ObjectPoolSystem

- 引擎底层嵌套实现对象池系统
- 针对Actor只需要实现IReusable，即可无感的使用**SpawnActor**从对象池创建actor & **DestroyActor**将actor回收到对象池
- 针对联机的Actor，也是花了一些时间去解决优化网络相关的问题
  - ActorChannel未关闭，导致客户端无法正常走到DestroyActor阶段(不会发Close Bunch)
  - ActorChannel回收
  - 因GuidCache导致客户端无法正常SpawnActor
  - 因GuidCache导致open过的Actor的ReplicatedComponents无法同步到客户端的问题
  - 客户端Actor被网络裁剪后，导致rep的子对象无法同步到客户端
- 同时还支持预加载，自动扩容等常规功能

（2）子弹使用对象池(例)

```
/**
 * Bullet base for this game.
 */
UCLASS(BlueprintType)
```

```
class GAME_API ABullet : public AGameProjectileBase, public IRelationshipInterface, public IReusable {
    /**
     * 对象池接口: OnSpawn
     */
    virtual void OnSpawn_Implementation() override;

    /**
    * 对象池接口: OnRecycle
    */
    virtual void OnRecycle_Implementation() override;
}
```

五、数据压缩

1. 网络数据类型压缩

（1）UE底层针对不同数据类型网络传输的处理

- **int8 / int16 / int32 / float / double**

按照实际类型长度的字节序列化，**无压缩**

```
// Used internally only to control the amount of generated code/type under control.
template<typename T>
FArchive& ByteOrderSerialize(T& Value)
{
   static_assert(!TIsSigned<T>::Value, "To reduce the number of template instances, cast 'Value' to a uint16&, uint32& or uint64& prior to the call or use ByteOrderSerialize(void*, int32).");

   if (!IsByteSwapping()) // Most likely case (hot path)
   {
      Serialize(&Value, sizeof(T));
      return *this;
   }
   return SerializeByteOrderSwapped(Value); // Slowest and unlikely path (but fastest than SerializeByteOrderSwapped(void*, int32)).
}
```

- **uint32**

提供两种压缩算法

```
/**
* 按实际占位长度序列化，但需要指定Max
*
* 只在一些特定场景使用(PackedVector/FixedFloat/NameIndex/)
*/
virtual void SerializeInt(uint32& Value, uint32 Max) override;

/**
* 按实际字节长度序列化(大部分场景使用)
*
* [0,     2^7 - 1]  1byte
* [2^7,   2^14 - 1] 2byte
* [2^14,  2^21 - 1] 3byte
* [2^21,  2^28 - 1] 4byte
* [2^28,  2^32 - 1] 5byte
*/
virtual void SerializeIntPacked(uint32& Value) override;
```

- **FString**

```
FArchive& operator<<( FArchive& Ar, FString& A )
{
   // > 0 for ANSICHAR, < 0 for UTF16CHAR serialization
   static_assert(sizeof(UTF16CHAR) == sizeof(UCS2CHAR), "UTF16CHAR and UCS2CHAR are assumed to be the same size!");

   if (Ar.IsLoading())
   {
       ...
   }
   else
   {
      A.Data.CountBytes(Ar);

      const bool bSaveUnicodeChar = Ar.IsForcingUnicode() || !FCString::IsPureAnsi(*A);
      if (bSaveUnicodeChar)
      {
         // Note: This is a no-op on platforms that are using a 16-bit TCHAR
         FTCHARToUTF16 UTF16String(*A, A.Len() + 1); // include the null terminator
         int32 Num = UTF16String.Length() + 1; // include the null terminator

         int32 SaveNum = -Num;
         Ar << SaveNum;
```

```

        if (Num)
        {
           if (!Ar.IsByteSwapping())
           {
              Ar.Serialize((void*)UTF16String.Get(), sizeof(UTF16CHAR) * Num);
           }
           else
           {
              TArray<UTF16CHAR> Swapped(UTF16String.Get(), Num);
              for (int32 CharIndex = 0; CharIndex < Num; ++CharIndex)
              {
                 Swapped[CharIndex] = ByteSwap(Swapped[CharIndex]);
              }
              Ar.Serialize((void*)Swapped.GetData(), sizeof(UTF16CHAR) * Num);
           }
        }
     }
     else
     {
        int32 Num = A.Data.Num();
        Ar << Num;

        if (Num)
        {
           Ar.Serialize((void*)StringCast<ANSICHAR>(A.Data.GetData(), Num).Get(), sizeof(ANSICHAR) * Num);
        }
     }
  }

  return Ar;
}
```

- **FName**

```
bool UPackageMap::StaticSerializeName(FArchive& Ar, FName& InName)
{
   if (Ar.IsLoading())
   {
       ...
   }
   else if (Ar.IsSaving())
   {
      const EName* InEName = InName.ToEName();
```

```
        uint8 bHardcoded = InEName && ShouldReplicateAsInteger(*InEName);
        Ar.SerializeBits(&bHardcoded, 1);
        if (bHardcoded && /* silence static analyzer */ InEName)
        {
            // send by hardcoded index
            checkSlow(InName.GetNumber() <= 0); // hardcoded names should never have a Number
            uint32 NameIndex = *InEName;
            Ar.SerializeIntPacked(NameIndex);
        }
        else
        {
            // send by string
            FString OutString = InName.GetPlainNameString();
            int32 OutNumber = InName.GetNumber();
            Ar << OutString << OutNumber;
        }
    }
    return true;
}
```

- **FText**

FText主要用于本地化多语言，因此相比FName和FString存在很多额外信息，就算一个Empty的FText在进行网络传输时也会占用100-200bit，因此**网络里是要严禁使用FText**

```
void FText::SerializeText(FStructuredArchive::FSlot Slot, FText& Value)
{
    ...
    const int32 OriginalFlags = Value.Flags;
    if(UnderlyingArchive.IsSaving())
    {
        Value.TextData->PersistText(); // We always need to do this when saving so that we can save the history correctly
        if(UnderlyingArchive.IsPersistent())
        {
            Value.Flags &= ~(ETextFlag::ConvertedProperty | ETextFlag::InitializedFromString); // Remove conversion flag before saving.
        }
    }
    Record << SA_VALUE(TEXT("Flags"), Value.Flags);
    ...
    if (UnderlyingArchive.IsSaving())
    {
        Value.Flags = OriginalFlags;
```

```
18    }
19
20    if (UnderlyingArchive.UE4Ver() >= VER_UE4_FTEXT_HISTORY)
21    {
22       bool bSerializeHistory = true;
23
24       if (UnderlyingArchive.IsSaving())
25       {
26          // Skip the history for empty texts
27          bSerializeHistory = !Value.IsEmpty() && !Value.IsCultureInvariant();
28
29          if (!bSerializeHistory)
30          {
31             int8 HistoryType = (int8)ETextHistoryType::None;
32             Record << SA_VALUE(TEXT("HistoryType"), HistoryType);
33
34             bool bHasCultureInvariantString = !Value.IsEmpty() &&
   Value.IsCultureInvariant();
35             Record << SA_VALUE(TEXT("bHasCultureInvariantString"),
   bHasCultureInvariantString);
36             if (bHasCultureInvariantString)
37             {
38                FString CultureInvariantString = Value.GetSourceString();
39                Record << SA_VALUE(TEXT("CultureInvariantString"),
   CultureInvariantString);
40             }
41          }
42       }
43       else if (UnderlyingArchive.IsLoading())
44       {
45           ...
46       }
47
48       if(bSerializeHistory)
49       {
50          FTextHistory& MutableTextHistory = Value.TextData-
   >GetMutableTextHistory();
51          MutableTextHistory.Serialize(Record);
52
53          if (Value.TextData->OwnsLocalizedString())
54          {
55             MutableTextHistory.SerializeForDisplayString(Record,
   Value.TextData->GetMutableLocalizedString());
56          }
57       }
58    }
59    ...
```

```
    if( Value.ShouldGatherForLocalization() )
    {
        UnderlyingArchive.ThisRequiresLocalizationGather();
    }
}
```

- **FVector / FRotator / FQuat**

```
// FVector无压缩
bool FVector::NetSerialize(FArchive& Ar, class UPackageMap* Map, bool& bOutSuccess)
{
    Ar << X;
    Ar << Y;
    Ar << Z;
    return true;
}

// FRotator(Pitch/Yaw/Roll[0, 360)), 使用了uint16进行压缩
bool FRotator::NetSerialize(FArchive& Ar, class UPackageMap* Map, bool& bOutSuccess)
{
    SerializeCompressedShort( Ar );
    bOutSuccess = true;
    return true;
}

// FQuat无压缩
bool FQuat::NetSerialize(FArchive& Ar, class UPackageMap*, bool& bOutSuccess)
{
    FQuat Q;

    if (Ar.IsSaving())
    {
        ...
    }

    Ar << Q.X << Q.Y << Q.Z;

    if ( Ar.IsLoading() )
    {
        ...
    }

    bOutSuccess = true;
```

```
    return true;
}
```

- 支持网络传输的UObject指针引用(包括UActor/UActorComponent)

① FNetworkGUID(最多32位)

```
friend FArchive& operator<<( FArchive& Ar, FNetworkGUID& G )
{
    Ar.SerializeIntPacked(G.Value);
    return Ar;
}
```

② ExportPathName(可选)

```
/**
 * This is the meat of the PackageMap class which serializes a reference to Object.
 */
bool UPackageMapClient::SerializeObject( FArchive& Ar, UClass* Class, UObject*& Object, FNetworkGUID *OutNetGUID)
{
    ...
    if (Ar.IsSaving())
    {
        ...
        // Write object NetGUID to the given FArchive
        InternalWriteObject( Ar, NetGUID, Object, TEXT( "" ), NULL );

        // If we need to export this GUID (its new or hasnt been ACKd, do so here)
        if (!NetGUID.IsDefault() && ShouldSendFullPath(Object, NetGUID))
        {
            check(IsNetGUIDAuthority());
            if ( !ExportNetGUID( NetGUID, Object, TEXT(""), NULL ) )
            {
                UE_LOG( LogNetPackageMap, Verbose, TEXT( "Failed to export in ::SerializeObject %s"), *Object->GetName() );
            }
        }

        return true;
    }
    else if (Ar.IsLoading())
```

```
    {
        ...
    }

    return true;
}
```

③ 属性同步

- **FStructProperty**

① 如果有实现了NetSerialize接口，会调用NetSerialize进行序列化

② 如果没有实现NetSerialize接口，默认按照UPROPERTY属性(且没标记NotReplicated)定义顺序BuildSharedData，然后进行序列化同步

③ 如果Struct嵌套Struct成员属性，循环继续上述过程

（2）数据类型同步压缩优化

- **float / double =》uintxx**

  浮点数在网络传输时是无法压缩的，因此可考虑根据实际情况转换成整型

  例如：float [0, 1] - 32bit，通过scale=100的倍率缩放改成, uint8 [0, 100] - 8bit

- **int8 / int16 / int32 / int64 =》uint8 / uint16 / uint32 / uint64**

  符号整型在网络传输时也是无法压缩的，因此也需要考虑换成对应长度的无符号整型

- **FText / FName / FString =》uintxx**

  FName和FString在UE里大多数情况都是wchar(双字节字符)，并且UE底层在传输FString类型未做任何压缩处理，如果传输频率较高，流量消耗也会很高

  例如：骨骼槽位BoneName可以改成BoneIndex

- **FQuat =》FVector =》FVector_NetQuantize / FVector_NetQuantize10 / FVector_NetQuantize100**

```
/**
 * FVector_NetQuantize
 *
 * 0 decimal place of precision.
 * Up to 20 bits per component.
 * Valid range: 2^20 = +/- 1,048,576
 *
 * Note: this is the historical UE format for vector net serialization
 *
 */
USTRUCT()
```

```
struct FVector_NetQuantize : public FVector

/**
 * FVector_NetQuantize10
 *
 * 1 decimal place of precision.
 * Up to 24 bits per component.
 * Valid range: 2^24 / 10 = +/- 1,677,721.6
 *
 */
USTRUCT()
struct FVector_NetQuantize10 : public FVector

/**
 * FVector_NetQuantize100
 *
 * 2 decimal place of precision.
 * Up to 30 bits per component.
 * Valid range: 2^30 / 100 = +/- 10,737,418.24
 *
 */
USTRUCT()
struct FVector_NetQuantize100 : public FVector
```

六、Tick

1. DS动态帧率优化

（1）前序

引擎层有提供动态修改帧率的接口，但实测下来不起效，通过研究引擎源码发现有很多会影响DS帧率的接口，其中*CVarMaxFPS*是导致不起效的真正原因，这个Console变量主要是表现层那边在调用（但因为DS比较难debug，因此具体原因目前还未排查到）

```
float UEngine::GetMaxTickRate(float DeltaTime, bool bAllowFrameRateSmoothing) const
{
   float MaxTickRate = 0;

   if (bAllowFrameRateSmoothing && IsAllowedFramerateSmoothing())
   {
      // Work in FPS domain as that is what the function will return.
      MaxTickRate = 1.f / RunningAverageDeltaTime;

      // Clamp FPS into ini defined min/ max range.
```

```
        if (SmoothedFrameRateRange.HasLowerBound())
        {
            MaxTickRate = FMath::Max( MaxTickRate, SmoothedFrameRateRange.GetLowerBoundValue() );
        }
        if (SmoothedFrameRateRange.HasUpperBound())
        {
            MaxTickRate = FMath::Min( MaxTickRate, SmoothedFrameRateRange.GetUpperBoundValue() );
        }
    }

    ...

    if (CVarUnsteadyFPS.GetValueOnAnyThread())
    {
        ...
        MaxTickRate = FMath::Clamp(LastMaxTickRate + RandDelta, 8.f, 32.f);
        ...
    }
    // 导致DS帧率不起效的可能性最大
    else if (CVarMaxFPS.GetValueOnAnyThread() > 0)
    {
        MaxTickRate = CVarMaxFPS.GetValueOnAnyThread();
    }

    return MaxTickRate;
}

float UGameEngine::GetMaxTickRate(float DeltaTime, bool bAllowFrameRateSmoothing) const
{
    float MaxTickRate = 0.f;

    ...

    // See if the code in the base class wants to replace this
    float SuperTickRate = Super::GetMaxTickRate(DeltaTime, bAllowFrameRateSmoothing);
    if(SuperTickRate != 0.0)
    {
        MaxTickRate = SuperTickRate;
    }

    return MaxTickRate;
}
```

（2）修改引擎层，强制应用DS的动态帧率

```
float UGameEngine::GetMaxTickRate(float DeltaTime, bool bAllowFrameRateSmoothing) const
{
   float MaxTickRate = 0.f;

   if (FPlatformProperties::SupportsWindowedMode() || IsRunningDedicatedServer())
   {
      // This applies for "non-console" platforms...

      ... other code

      if( World )
      {
         UNetDriver* NetDriver = World->GetNetDriver();
         // In network games, limit framerate to not saturate bandwidth.
         if( NetDriver && (NetDriver->GetNetMode() == NM_DedicatedServer || (NetDriver->GetNetMode() == NM_ListenServer && NetDriver->bClampListenServerTickRate)))
         {
            // We're a dedicated server, use the LAN or Net tick rate.
            MaxTickRate = FMath::Clamp( NetDriver->NetServerMaxTickRate, 1, 1000 );

            // Force to apply 'NetDriver->NetServerMaxTickRate' on dedicated server
            if (NetDriver->bForceToApplyNetServerMaxTickRate &&
               (IsRunningDedicatedServer() || NetDriver->GetNetMode() == NM_DedicatedServer))
            {
               return MaxTickRate;
            }
         }
      }
   }

   // See if the code in the base class wants to replace this
   float SuperTickRate = Super::GetMaxTickRate(DeltaTime, bAllowFrameRateSmoothing);
   if(SuperTickRate != 0.0)
   {
      MaxTickRate = SuperTickRate;
   }

```

```
    return MaxTickRate;
}
```

（3）DS动态帧率调整

- **动态调整接口**

从GS发送的*CmdLine*中获取*ServerTickRate*参数，因此**可根据线上实际情况动态调整**

```
bool USeasunNetDriver::InitListen(FNetworkNotify* InNotify, FURL& ListenURL, bool bReuseAddressAndPort, FString& Error) {
    const bool bRet = Super::InitListen(InNotify, ListenURL, bReuseAddressAndPort, Error);

    // DS InitListen阶段动态调整DS帧率
    if (this->IsServer()) {
        ApplyConsoleServerTickRate();
    }

    return bRet;
}

void USeasunNetDriver::ApplyConsoleServerTickRate() {
    int32 ConsoleNetServerMaxTickRate = -1;
    const bool bHasConsoleVar = FParse::Value(FCommandLine::Get(), TEXT("ServerTickRate="), ConsoleNetServerMaxTickRate);

    if (bHasConsoleVar && ConsoleNetServerMaxTickRate > 0 && ConsoleNetServerMaxTickRate != this->NetServerMaxTickRate) {
        this->NetServerMaxTickRate = ConsoleNetServerMaxTickRate;
    }
}
```

- **DS帧率默认从30调整到15**

```
[/Script/OnlineSubsystemUtils.IpNetDriver]
NetServerMaxTickRate=15
bForceToApplyNetServerMaxTickRate=true
```

（4）优化效果

内网测试数据：

- 单核CPU占用率

[30帧]　　[20帧]　　[15帧]

41.9% => 34.5% => 28.5%

七、Actor/ActorComponent专项

1、SetReplicates(false)

当不需要网络同步时，关闭同步

2、NetUpdateFrequency

网络同步频率，针对同步需求小的Actor可降低同步频率

3、Relevancy

网络相关性，通常除非特别需要，bAwaysRelevant设为false

4、NetCullDistanceSquared

网络裁剪距离，根据实际情况进行调整

例：Bunker等可交互物动态生成，可以优化客户端Actor数量 (闫乐)

5、Reliable RPC

客户端效果相关的RPC原则上一律采用Unreliable，影响逻辑的再用Reliable

```
void UChannel::ReceivedRawBunch( FInBunch & Bunch, bool & bOutSkipAck )
{
    ...
    if ( Bunch.bReliable && Bunch.ChSequence != Connection->InReliable[ChIndex] + 1 )
    {
        ...
        if ( NumInRec >= RELIABLE_BUFFER )
        {
            Bunch.SetError();
            UE_LOG( LogNetTraffic, Error, TEXT( "UChannel::ReceivedRawBunch: Too many reliable messages queued up" ) );
            return;
        }

        checkSlow(NumInRec<=RELIABLE_BUFFER);
        //AssertInSequenced();
    }
    ...
}

```

```
void UNetConnection::ReceivedPacket( FBitReader& Reader, bool bIsReinjectedPacket)
{
    ...
    // Dispatch the raw, unsequenced bunch to the channel.
    bool bLocalSkipAck = false;
    Channel->ReceivedRawBunch( Bunch, bLocalSkipAck ); //warning: May destroy channel.
    ...
    // Disconnect if we received a corrupted packet from the client (eg server crash attempt).
    if ( !Driver->ServerConnection && ( Bunch.IsCriticalError() || Bunch.IsError() ) )
    {
       UE_LOG( LogNetTraffic, Error, TEXT("Received corrupted packet data from client %s.  Disconnecting."), *LowLevelGetRemoteAddress() );
       Close();
       return;
    }
    ...
}
```

6、子对象(DefaultSubObject)

子对象(DefaultSubObject)作为UProperty属性时，如果bReplicates设为true，就不需要选择Replicated

```
UPROPERTY(Transient /*, Replicated*/ )
class UTeamAbilityComponent* TeamAbilityComponent;
```

引擎层的处理：

- ReplicateProperties：Actor属性同步
- ReplicateSubobjects：子对象同步

如果Actor子对象(ActorComponent)被标记为Replicate的属性，且本身打开了网络同步，就会被视为子对象和属性同步两次

```
int64 UActorChannel::ReplicateActor()
{
    ...
    if (!bIsNewlyReplicationPaused)
```

```
    {
        {
            UE_NET_TRACE_OBJECT_SCOPE(ActorReplicator->ObjectNetGUID, Bunch, GetTraceCollector(Bunch), ENetTraceVerbosity::Trace);
            // 属性同步
            WroteSomethingImportant |= ActorReplicator->ReplicateProperties(Bunch, RepFlags);
        }

        // 子对象同步
        WroteSomethingImportant |= Actor->ReplicateSubobjects(this, &Bunch, &RepFlags);
        ...
    }
    ...
}
```

7、StaticMeshComponent

Mesh组件不需要勾选bReplicates，可以在业务层Actor的Native代码中强制关闭，避免策划随意乱改

```
void AGameSkillInteractorBase::HandlePerformance_DS() const {
   // 关闭StaticMeshComp的网络同步
   TArray<UActorComponent*> StaticMeshComps;
   GetComponents(UStaticMeshComponent::StaticClass(), StaticMeshComps);
   if (StaticMeshComps.Num() > 0) {
      for (UActorComponent* StaticMeshComp: StaticMeshComps) {
         if (IsValid(StaticMeshComp)) {
            StaticMeshComp->SetIsReplicated(false);
         }
      }
   }
   ...
}
```

8、关闭骨骼更新

如果Actor不需要骨骼更新，建议在Native代码中直接关闭，在蓝图中按需打开

```
void AGameSkillInteractorBase::HandlePerformance_DS() const {
   ...
   // 去掉骨骼更新
```

```
    USkeletalMeshComponent* SkeletalMeshComponent = Cast<USkeletalMeshComponent>(GetComponentByClass(USkeletalMeshComponent::StaticClass()));
    if (SkeletalMeshComponent) {
        SkeletalMeshComponent->VisibilityBasedAnimTickOption = CVarEnableBonesUpdateOnDS.GetValueOnGameThread() ?
            EVisibilityBasedAnimTickOption::AlwaysTickPoseAndRefreshBones:
            EVisibilityBasedAnimTickOption::AlwaysTickPose;
    }
    ...
}
```

9、关闭移动

不需要移动的，关闭移动组件Tick/Replicate

```
void AGameSkillInteractorBase::HandlePerformance_DS() const {
    ...
    // 关闭移动组件
    if (GetCharacterMovement() != nullptr) {
        GetCharacterMovement()->SetIsReplicated(false);
        GetCharacterMovement()->SetComponentTickEnabled(false);
        GetCharacterMovement()->Deactivate();
    }
}
```